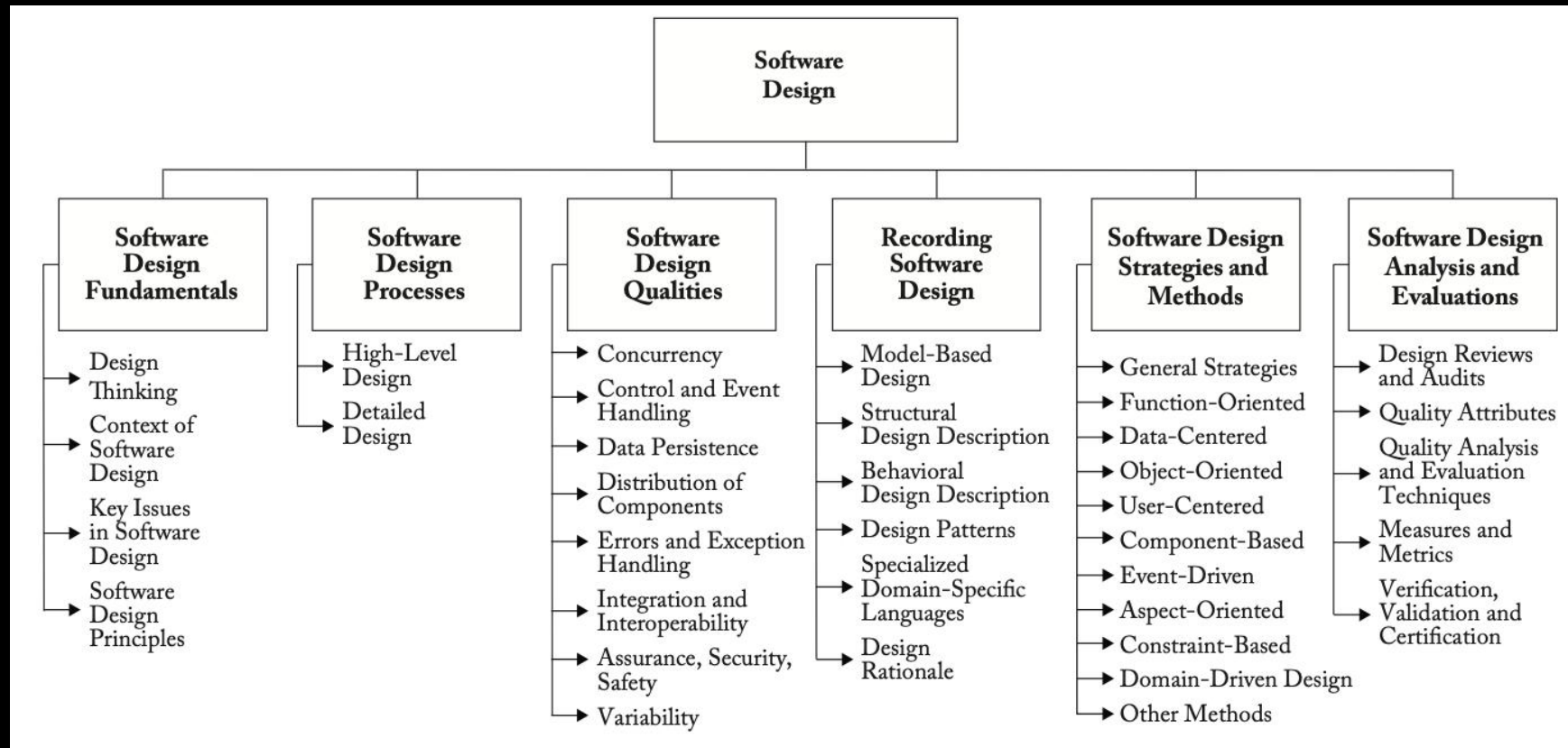


Object– Oriented Software Engineering

TTTE6104

Software Engineering
Paradigm



Introduction

- **Definition of OOSE**
 - **A systematic approach to software development that incorporates object-oriented principles, methodologies, and techniques to design and implement software systems.**
- **Importance of OOSE in software development**
 - **Focuses on modeling real-world entities as "objects," which encapsulate both data (attributes) and behavior (methods) related to those entities**

Key Components of OOSE

- **Object–Oriented Principles**

- **Encapsulation:** Bundling data and methods that operate on that data within a single unit (the object), promoting data hiding and abstraction.
- **Inheritance:** Allowing a new class to inherit properties and behaviors from an existing class, facilitating reusability and the creation of hierarchical relationships.
- **Polymorphism:** Enabling objects to be treated as instances of their parent class, allowing for method overriding and overloading, which enhances flexibility in code execution.

- **Modeling Real–World Systems**

- OOSE uses models to represent real–world entities and their interactions. These models help in understanding and specifying the requirements of the software system.

Key Components of OOSE

- **Software Development Life Cycle**
 - OOSE integrates the principles of OOP throughout all phases of the SDLC, including requirements gathering, analysis, design, implementation, testing, and maintenance, ensuring a cohesive approach to software development.
- **Unified Modeling Language (UML)**
 - OOSE often employs UML as a standardized modeling language to visualize the design of a system, facilitating communication among stakeholders.
- **Focus on Reusability and Maintainability**
 - By emphasizing modular design and the use of classes and objects, OOSE promotes code reusability and simplifies maintenance, which are crucial for long-term software sustainability.

Software Development Life Cycle (SDLC)

1. Requirement Analysis
 2. System Design
 3. Implementation
 4. Testing
 5. Deployment
 6. Maintenance
- **Role of OOSE in each phase**

Object–Oriented Analysis and Design (OOAD)

- A way to design software by thinking of everything as objects similar to real–life things
- Two distinct but closely related processes: Object–Oriented Analysis (OOA) and Object–Oriented Design (OOD)

Important Aspects of OOAD

- Object–Oriented Programming: In this the real–world items are represented/mapped as software objects with attributes and methods that relate to their actions.
- Design Patterns: Design patterns are used by OOAD to help developers in building software systems that are more efficient and maintainable.
- UML Diagrams: UML diagrams are used in OOAD to represent the different components and interactions of a software system.
- Use Cases: OOAD uses use cases to help developers understand the requirements of a system and to design software systems that meet those requirements.

Benefits of Object–Oriented Analysis and Design (OOAD)

- It increases the modularity and maintainability of software by encouraging the creation of tiny, reusable parts that can be combined to create more complex systems.
- It provides a high–level, abstract representation of a software system, making understanding and maintenance easier.
- It promotes object–oriented design principles and the reuse of objects, which lowers the amount of code that must be produced and raises the quality of the program.
- Software engineers can use the same language and method that OOAD provides to communicate and work together more successfully in groups.
- It can assist developers in creating scalable software systems that can adapt to changing user needs and business demands over time.

Real world applications of OOAD

- Banking Software
- Electronic Health Record (EHR) Systems
- Flight Control Systems
- Telecom Billing Systems
- Online Shopping Platforms

Class Activity

- Work in teams to identify a real-world application of Object-Oriented Analysis and Design, analyze its components, and present the findings.
- Each team will select a real-world example of a software application that utilizes Object-Oriented Analysis and Design principles.
- Suggested sources – Academic Articles, Whitepapers, Case Studies, Open Source Projects (GitHub)

- Each team will analyze their selected application and gather the following information:
- **Project Title:** Name of the application or system.
- **Overview:** Brief description of the application and its purpose.
- **Diagrams:** Create and include relevant UML diagrams such as:
 - Use Case Diagram
 - Class Diagram
 - Sequence Diagram
- **Tools Used:** Identify the tools and technologies used in the development of the application (e.g., programming languages, frameworks, design tools).
- **Benefits of Using OO Approach:** Discuss the advantages of using an object-oriented approach in their chosen example, including aspects like modularity, maintainability, scalability, and ease of collaboration.